

The Mathematics and Design of DuskCrayfish

Strawberry Creek Monitoring Group

What the model is doing

DuskCrayfish learns what the creek normally looks like and then flags moments it cannot predict well. Everything below is in service of one idea: take a window of recent sensor history across all the connected sites, and predict what the very next reading should be at every site. When the real reading and the prediction diverge sharply on conductivity, that gap is the anomaly signal. The model is never told what a spill looks like. It only ever learns normal, and an anomaly is whatever it fails to reproduce.

This document explains the math and, alongside each piece, why the model is built the way it is. The architecture is deliberately small and the choices behind that are part of the story, not an afterthought.

The deployed configuration

The values below are the actual settings the model runs with, drawn from the project’s configuration. They are referenced throughout the math that follows so the general equations stay tied to this specific model.

Setting	Value
Sequence length T	24 timesteps (6 hours at 15 min sampling)
Features per node F	6 when weather is present
Hidden dimension d	16
Graph convolution layers	1
LSTM layers	1
Graph convolution type	GCN
Dropout	0.1
Training epochs	10 (early stopping, patience 5)
Batch size	128
Learning rate	0.001
Train / validation split	0.8 / 0.2
Threshold percentile	99
Rain lookback window	12 hours
Rain threshold multiplier m	2.0
Rain amount that counts as wet θ	0.1
Imputation gap limit	24 hours

A word on why these are small. One graph layer, one LSTM layer, and a hidden dimension of only 16 is a deliberately lean model. The training data is a rolling window of recent creek behavior,

not a vast historical archive, and a larger model on that much data would overfit, memorizing the recent past rather than learning the creek’s general rhythm. A small model also trains in seconds on a CPU, which matters because the data is a moving window and the model is retrained regularly to stay current rather than trained once and frozen. The choices are matched to the data that exists, not to what would look impressive on paper.

Setting up the data

The creek is represented as a graph with N sensor nodes whose connections follow the direction water flows. At each timestep every node carries a vector of features: conductivity, depth, and temperature from the sensors, plus air temperature, rainfall, and solar radiation merged in from weather data when it is available. The model does not hardcode the number of features. It reads the feature count from the data it is given at runtime and sizes its input and output layers to match, which is why it runs with six features when weather is present and adapts cleanly when a channel is missing. We write F for that count and treat $F = 6$ as the normal weather-present case throughout.

A single training example is a window of $T = 24$ consecutive timesteps, written

$$\mathbf{X} \in \mathbb{R}^{T \times N \times F},$$

so $\mathbf{X}_t \in \mathbb{R}^{N \times F}$ is the snapshot of every node’s features at timestep t . The target is the snapshot immediately after the window,

$$\mathbf{Y} = \mathbf{X}_{T+1} \in \mathbb{R}^{N \times F}.$$

The choice of $T = 24$ is six hours of history at the creek’s fifteen minute cadence. That is long enough to capture the daily rhythms the creek actually has, the warming through the afternoon, the overnight runoff from sprinklers, without reaching so far back that old conditions drown out the recent trend the model is trying to extend.

Before anything else, every feature is normalized to a z-score using statistics from complete data only,

$$z = \frac{x - \mu}{\sigma}.$$

This choice has a consequence the model leans on. After normalization a value of zero is the average, the neutral middle of the distribution. That is exactly the placeholder used for missing sensor readings, so a gap contributes nothing rather than dragging the model up or down. This is what lets the model run on a network where some sites are offline, a footbridge sensor that is down simply reads as the neutral zero everywhere it is missing.

Step one, the spatial step with a graph convolution

For each timestep, each node’s features are informed by the nodes connected to it, because a contaminant upstream shows up downstream as the water carries it. With $\mathbf{H} \in \mathbb{R}^{N \times F}$ the node features at one timestep, the graph convolution computes

$$\mathbf{H}' = \text{ReLU}\left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H} \mathbf{W}\right),$$

where $\mathbf{W} \in \mathbb{R}^{F \times d}$ is a learned weight matrix projecting the F features into the hidden size $d = 16$, and the nonlinearity is

$$\text{ReLU}(x) = \max(0, x).$$

This is the standard graph convolutional operator. The adjacency with inserted self loops is $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, and $\hat{\mathbf{D}}$ is its diagonal degree matrix with $\hat{D}_{ii} = \sum_j \hat{A}_{ij}$. The self loops let each node keep its own features rather than only seeing neighbors, and the degree scaling $\hat{\mathbf{D}}^{-1/2}$ on both sides keeps a heavily connected site from swamping a lightly connected one. The graph library adds the self loops and applies this normalization automatically, so the edge list the model is given does not need to include them.

The edges for the deployed five-node core

It is worth making this concrete for the network the model actually runs on. The five active nodes, in the order the code indexes them, are footbridge, north fork 0, south fork 2, south fork 1, and oxford. The connections are defined as directed edges following the physical direction of water flow downhill. There are four of them:

$$\begin{aligned} \text{north fork 0} &\rightarrow \text{footbridge}, & \text{footbridge} &\rightarrow \text{oxford}, \\ \text{south fork 1} &\rightarrow \text{south fork 2}, & \text{south fork 2} &\rightarrow \text{oxford}. \end{aligned}$$

In words, the north branch runs north fork 0 into footbridge into oxford, the south branch runs south fork 1 into south fork 2 into oxford, and the two branches converge at oxford. Writing these as a directed adjacency $\mathbf{A}$, where entry (i, j) is one when there is an edge from node i to node j , in the code's index order (footbridge, north fork 0, south fork 2, south fork 1, oxford) gives

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Reading the rows: footbridge flows to oxford, north fork 0 flows to footbridge, south fork 2 flows to oxford, south fork 1 flows to south fork 2, and oxford flows nowhere, since it is the most downstream point. That last empty row is the signature of a sink, the confluence where the network ends. The graph convolution then adds a self loop to every node and applies the degree normalization above, so each node's update blends its own value with the values flowing into it along these edges. The structure is sparse and entirely determined by the creek's physical layout, which is the point: the model does not have to learn who connects to whom, it is told, and told correctly.

What the convolution does to a single node

The matrix form is compact but hides what happens to one site. Writing it per node, the new representation of node i is a normalized sum over itself and the nodes connected to it,

$$\mathbf{h}'_i = \text{ReLU} \left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\hat{d}_i \hat{d}_j}} \mathbf{h}_j \mathbf{W} \right),$$

where $\mathcal{N}(i)$ is the set of nodes connected to i through the graph and $\hat{d}_i$ is node i 's degree including its self loop. This is the precise statement of borrow from your neighbors: each node's updated features are a degree-weighted blend of its own and its neighbors', passed through the learned projection $\mathbf{W}$ and the nonlinearity. A spike in conductivity at a site therefore enters the connected site's representation on the very next layer, which is exactly the propagation the model needs to recognize a real event as it moves through the creek.

Why these choices

Two design choices live here. The model uses a graph convolution of the GCN type, set explicitly in the configuration. A GCN weights neighbors by the fixed graph structure, which is the right prior when the connections are real water flow rather than relationships the model should infer. The flow topology is known physics, so there is little reason to make the model learn neighbor weightings it would only overfit on limited data. The model also uses a single graph layer. One layer means each node sees its immediate neighbors. On a network this small, with short flow paths, one hop already reaches the sites that matter, and stacking more layers would blur every node toward a single averaged value, a known failure of deep graph networks called oversmoothing.

Step two, collapsing the network into one creek state

The temporal model that comes next wants a single vector for the whole creek at each timestep, not N separate node vectors, so we average across the nodes,

$$\mathbf{h}_t = \frac{1}{N} \sum_{i=1}^N \mathbf{H}'_i \in \mathbb{R}^d.$$

Doing this at every timestep produces a sequence $\mathbf{h}_1, \dots, \mathbf{h}_T$, each entry a summary of the whole connected network at one moment. Mean pooling rather than a more elaborate combination is the simple, robust choice, and it interacts well with the missing-data handling: a node reading the neutral zero contributes nothing to the average, so the creek summary naturally reflects only the sites that have real data.

Step three, the temporal step with an LSTM

The sequence of creek summaries goes into a single layer Long Short Term Memory network, which tracks how the signal evolves and carries relevant information forward across steps. At each timestep it maintains a hidden state $\mathbf{h}_t^{\text{lstm}}$ and a cell state $\mathbf{c}_t$, updated through gates:

$$\begin{aligned} \mathbf{f}_t &= \sigma(\mathbf{W}_f \mathbf{h}_t + \mathbf{U}_f \mathbf{h}_{t-1}^{\text{lstm}} + \mathbf{b}_f), \\ \mathbf{i}_t &= \sigma(\mathbf{W}_i \mathbf{h}_t + \mathbf{U}_i \mathbf{h}_{t-1}^{\text{lstm}} + \mathbf{b}_i), \\ \mathbf{o}_t &= \sigma(\mathbf{W}_o \mathbf{h}_t + \mathbf{U}_o \mathbf{h}_{t-1}^{\text{lstm}} + \mathbf{b}_o), \\ \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c \mathbf{h}_t + \mathbf{U}_c \mathbf{h}_{t-1}^{\text{lstm}} + \mathbf{b}_c), \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \\ \mathbf{h}_t^{\text{lstm}} &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \end{aligned}$$

The three gates are each squashed by the sigmoid σ into values between zero and one, acting as soft switches. The forget gate $\mathbf{f}_t$ decides how much of the previous memory to keep, the input gate $\mathbf{i}_t$ how much of the new candidate $\tilde{\mathbf{c}}_t$ to store, and the output gate $\mathbf{o}_t$ how much of the resulting memory to emit. The cell update line is where the network actually remembers or forgets. The symbol $\odot$ is elementwise multiplication, and the $\mathbf{W}$, $\mathbf{U}$, and $\mathbf{b}$ are learned.

The LSTM was chosen over a plain recurrent network because its gates let it hold a trend from hours earlier when relevant and discard it when not, which matters for a six hour window. A single LSTM layer is used for the same reason the rest of the model is small: the sequences are short and the data is limited, so one layer captures the temporal pattern without overfitting. We keep the final hidden state,

$$\mathbf{h}_T^{\text{lstm}} \in \mathbb{R}^d,$$

the model’s compressed understanding of how the creek has been behaving over the whole window.

Step four, predicting the next reading

We expand that summary back to all N nodes and pass it through a learned linear layer,

$$\hat{\mathbf{Y}}_i = \mathbf{W}_{\text{out}} \mathbf{h}_T^{\text{lstm}} + \mathbf{b}_{\text{out}}, \quad i = 1, \dots, N,$$

with $\mathbf{W}_{\text{out}} \in \mathbb{R}^{F \times d}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^F$ learned, giving a predicted feature vector for each node and the full prediction $\hat{\mathbf{Y}} \in \mathbb{R}^{N \times F}$. Because the output layer is sized to F , the same feature count the input was sized to, the model predicts every channel it was given, which is what the detection step draws on next.

Training, learning normal

Training minimizes the mean squared error between prediction and truth,

$$\mathcal{L} = \frac{1}{NF} \sum_{i=1}^N \sum_{j=1}^F \left(\hat{\mathbf{Y}}_{ij} - \mathbf{Y}_{ij} \right)^2,$$

over training data that contains only ordinary creek behavior. The model learns the absence of spills, not spills themselves.

Two training choices are worth naming. Dropout of 0.1 is applied during training, randomly zeroing a small fraction of hidden units each step, which prevents the small model from leaning too hard on any single pattern. Training runs for up to 10 epochs with early stopping at patience 5, meaning if the validation loss stops improving for five epochs the run halts and the best weights are restored, so the model does not train past the point where it is still learning something general. Training is done in plain 32 bit precision on CPU, because the mixed precision that speeds up large models on a graphics card actually breaks the LSTM computation on a CPU and offers no benefit at this size.

Detection, model all and alert one

Once trained, the model runs on new data and we examine where it missed. The squared prediction error at node i on feature j is

$$e_{ij} = \left(\hat{\mathbf{Y}}_{ij} - \mathbf{Y}_{ij} \right)^2.$$

Here is the defining choice. The model predicts all features, but only conductivity error is used to decide whether something is anomalous. This is the model all, alert one strategy. Conductivity is the channel that actually responds to spills, while the others, rainfall especially, produce large prediction errors that have nothing to do with water quality. An earlier version that scored on the average error across all features was effectively flagging rainstorms, because the rain prediction error swamped everything. Isolating conductivity removed those false alarms. With c the conductivity index, the anomaly score is

$$s = \frac{1}{N} \sum_{i=1}^N e_{ic},$$

the conductivity error averaged across the nodes. Averaging across nodes is itself a choice grounded in physics: a deviation appearing at several connected sites at once scores higher than a single site blip, because a real event propagates through the network while sensor noise stays local.

A timestep is flagged when its score exceeds a threshold τ , fixed once during training as the 99th percentile of the conductivity errors on held out validation data,

$$\tau = \text{percentile}_{99}(\{s^{\text{val}}\}).$$

Fixing it at training time rather than recomputing it on whatever data is in front of the model is deliberate, recomputing would be circular and would let the threshold drift with each batch. The 99th percentile means that under normal conditions only about one percent of timesteps would cross it, which sets the false alarm rate before deployment rather than discovering it after.

Detection, the rain adjustment

Rain genuinely raises conductivity without being a spill, as runoff carries road salt and grime into the creek. To avoid alarming on ordinary rain, the threshold is raised during wet periods. Summing rainfall over the preceding 12 hour window, the per timestep threshold is

$$\tau_t = \begin{cases} \tau & \text{if } \sum_{k \in \text{window}} r_k \leq \theta, \\ m \cdot \tau & \text{if } \sum_{k \in \text{window}} r_k > \theta, \end{cases}$$

with r_k the rainfall at timestep k , $\theta = 0.1$ the small amount that counts as wet, and $m = 2.0$ the multiplier. In plain terms, after recent rain the model becomes twice as skeptical of a conductivity deviation before calling it a spill. The 12 hour window and the doubling are the tuned values; they suppress the routine first flush of a storm while still leaving room to catch a deviation large enough to clear twice the normal bar.

The honest limit of this rule is built into its shape. The threshold is only raised while rain is recent, so a delayed first flush pulse that arrives well after the rain has stopped falls back under the ordinary threshold and can be flagged. Whether that is correct depends on whether a delayed

runoff event is worth knowing about, which is a judgment about what the system should alarm on rather than a flaw in the math.

What the choices add up to, and what they leave out

The honest summary of the design is that nearly every choice points the same direction: keep the model small and matched to limited, rolling data, encode the known physics of water flow directly into the graph rather than making the model learn it, and decide anomalies on the one channel that actually carries the signal. That focus is also where the limitations come from. The features are the sensor and weather channels with no time of year information, so the model has no seasonal context and reads an out of season window as mildly unusual everywhere. Scoring on conductivity alone means an anomaly that shows up only in another channel, such as a dissolved oxygen crash from an oil sheen, would not be caught by the current alarm. And the rain adjustment, as above, misses delayed first flush events. Each of these is a direct consequence of a choice made for a good reason, which is why the natural next steps, adding cyclical time features, scoring additional channels as sensors come online, and widening the post rain window, are extensions of the existing design rather than replacements of it.